

AI Project code

Project: ToxiBERT

TrainModel

```
import pandas as pd

import torch

from transformers import DistilBertTokenizer, DistilBertForSequenceClassification

from sklearn.model_selection import train_test_split

from torch.utils.data import Dataset, DataLoader

from tqdm import tqdm


# Load dataset


# Dataset is not uploaded on github because the size is too large. so i will provide the
dataset link of the you can go on that and download the dataset

# link: https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge/data

# You can download the dataset from Kaggle: https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge/data

df = pd.read_csv('jigsaw-toxic-comment-train-processed-seqlen128.csv')

label_cols = ['toxic', 'severe_toxic', 'obscene', 'threat', 'insult', 'identity_hate']

df = df.dropna(subset=label_cols)

df = df.sample(n=10000, random_state=42).reset_index(drop=True)

df['labels'] = df[label_cols].values.tolist()


# Split data

train_texts, val_texts, train_labels, val_labels = train_test_split(
```

```
list(df['comment_text']), df['labels'].tolist(), test_size=0.2, random_state=42
)
```

```
tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')
train_enc = tokenizer(train_texts, truncation=True, padding=True, max_length=128)
val_enc = tokenizer(val_texts, truncation=True, padding=True, max_length=128)
```

```
class ToxicDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.float32)
        return item

    def __len__(self):
        return len(self.labels)
```

```
train_dataset = ToxicDataset(train_enc, train_labels)
val_dataset = ToxicDataset(val_enc, val_labels)
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=8)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = DistilBertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased', num_labels=6, problem_type="multi_label_classification"
).to(device)
```

```
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
```

```
# Training loop
```

```
for epoch in range(2):
```

```
    model.train()
```

```
    loop = tqdm(train_loader, desc=f"Epoch {epoch+1}")
```

```
    for batch in loop:
```

```
        batch = {k: v.to(device) for k, v in batch.items()}
```

```
        optimizer.zero_grad()
```

```
        outputs = model(**batch)
```

```
        loss = outputs.loss
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
        loop.set_postfix(loss=loss.item())
```

```
# Save model and tokenizer
```

```
model.save_pretrained('./model')
```

```
tokenizer.save_pretrained('./tokenizer')
```

```
torch.save(model.state_dict(), './model/model.pt')
```

APP.py

```
from flask import Flask, render_template, request, jsonify
```

```

import torch

from transformers import DistilBertTokenizer, DistilBertForSequenceClassification

import numpy as np

app = Flask(__name__)

# Load model and tokenizer

model = DistilBertForSequenceClassification.from_pretrained('./model')

model.load_state_dict(torch.load('./model/model.pt', map_location=torch.device('cpu')))

model.eval()

tokenizer = DistilBertTokenizer.from_pretrained('./tokenizer')

LABELS = ['toxic', 'severe_toxic', 'obscene', 'threat', 'insult', 'identity_hate']

@app.route('/')
def index():

    print(" 🟢 Your toxic comment detector app is running!")

    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():

    comment = request.form['comment']

    inputs = tokenizer(comment, return_tensors='pt', truncation=True, padding=True,
max_length=128)

    with torch.no_grad():

        outputs = model(**inputs)

```

```

logits = outputs.logits
probs = torch.sigmoid(logits).numpy()[0]

result = {label: round(float(prob), 2) for label, prob in zip(LABELS, probs)}
return jsonify(result)

import os

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))

    print(f"🔒 HTTPS Enabled! Visit https://192.168.x.x:{port} from other devices on the
    same network.")

    context = ('cert.pem', 'key.pem') # SSL certificate and key
    app.run(debug=False, host='0.0.0.0', port=port, ssl_context=context)

```

Voice_input.js

```

function voiceInput() {

    const recognition = new (window.SpeechRecognition ||
window.webkitSpeechRecognition)();

    recognition.lang = 'en-US';

    recognition.start();

    recognition.onresult = function (event) {

        const transcript = event.results[0][0].transcript;

        document.querySelector('textarea[name="comment"]').value = transcript;
    }
}

```

```
document.getElementById('predict-form').dispatchEvent(new Event('submit'));  
  
};  
  
recognition.onerror = function () {  
    alert("Voice input failed. Please try again.");  
};  
}
```

Voice_Output.js

```
function speakOutput(text) {  
    if (!text) return;  
    const utterance = new SpeechSynthesisUtterance(text);  
    utterance.onend = () => {};  
    speechSynthesis.cancel(); // Stop previous if any  
    speechSynthesis.speak(utterance);  
}
```

Index.html

```
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>
```

```
<meta charset="UTF-8" />

<meta name="viewport" content="width=device-width, initial-scale=1.0"/>

<title>Toxic Comment Detector</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet"/>

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<script src="/static/voice_input.js" defer></script>

<script src="/static/voice_output.js" defer></script>

<style>

  body {

    background: linear-gradient(to right, #f9f9f9, #e1f5fe);

    font-family: 'Segoe UI', sans-serif;

    padding-bottom: 100px;

  }

  .container {

    max-width: 700px;

    background: white;

    border-radius: 15px;

    padding: 30px;

    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.1);

  }

  .btn-custom {

    border-radius: 20px;

  }

  .emoji {

    font-size: 1.5rem;

  }

  #result-list li {
```

```
    font-size: 1.1rem;
}

.mic, .speak {
    cursor: pointer;
    margin-left: 10px;
}

#loading {
    font-style: italic;
    color: #555;
}

footer {
    position: fixed;
    bottom: 0;
    width: 100%;
    background: #1e3c72;
    color: white;
    padding: 10px;
    text-align: center;
    box-shadow: 0 -1px 5px rgba(0,0,0,0.1);
}

#chart-container canvas {
    margin-top: 20px;
}

#barChart {
width: 100%;
max-width: 400px;
height: 300px;
}
```

```
#pieChart {  
  width: 100%;  
  max-width: 300px;  
  height: 300px;  
}
```

```
.chart-container {  
  flex: 1 1 300px;  
  max-width: 100%;  
  height: auto;  
}
```

```
#barChartContainer {  
  min-width: 650px; /* Wider than pieChart */  
}
```

```
#pieChartContainer {  
  min-width: 550px;  
}
```

```
@media (max-width: 768px) {  
  #barChartContainer, #pieChartContainer {  
    min-width: 100%;  
  }  
}
```

```
.chart-canvas {  
  width: 100% !important;
```

```
height: 600px !important;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h2 class="text-center mb-4">🔍 Toxic Comment Classifier</h2>
```

```
<form id="predict-form">
```

```
<div class="mb-3">
```

```
<label>Speak your Comment </label>
```

```
<textarea class="form-control" name="comment" rows="3" placeholder="Type or speak..." required></textarea>
```

```
</div>
```

```
<button type="submit" class="btn btn-primary btn-custom" id="check-btn">Check</button>
```

```
<button type="button" id="new-check" class="btn btn-secondary btn-custom ms-2" style="display:none;">Check Another Comment</button>
```

```
</form>
```

```
<div id="loading" class="mt-3" style="display:none;">⌚ Please wait... analyzing the comment...</div>
```

```
<div id="result" class="mt-4"></div>
```

```
</div>
```

```
<div id="suggestions" class="mt-4" style="display:none;">
```

```
<div class="alert alert-info">
```

💡 Want to see prediction history or a category-wise chart?

```
<div class="d-flex justify-content-end mt-3">  
  <button class="btn btn-outline-info me-2" id="show-history">Show History</button>  
  <button class="btn btn-outline-info" id="toggle-chart">Show Charts</button>  
</div>  
  
<div id="chart-container" class="mt-3" style="display:none;">  
  <!-- <canvas id="chart" width="400" height="300"></canvas> -->  
</div>
```

```
<div class="d-flex justify-content-around flex-wrap">  
  <div id="barChartContainer" class="chart-container">  
    <canvas id="barChart" class="chart-canvas"></canvas>  
  </div>  
  <div id="pieChartContainer" class="chart-container">  
    <canvas id="pieChart" class="chart-canvas"></canvas>  
  </div>  
</div>
```

```
</div>  
</div>  
</div>
```

```
<footer>
```

💡 Developed by Arun Kumar & Ayushi Saran Singh

✉ for any query Contact on : thakurarunkumar200411@gmail.com

© Copyright :All right reserved by Toxic Comment Classifier-2025

```
</footer>
```

```
<script>
```

```
const form = document.getElementById('predict-form');
const resultBox = document.getElementById('result');
const loading = document.getElementById('loading');
const newCheckBtn = document.getElementById('new-check');
const checkBtn = document.getElementById('check-btn');
const textArea = document.querySelector('textarea');
const suggestions = document.getElementById('suggestions');
const chartToggleBtn = document.getElementById('toggle-chart');
const chartContainer = document.getElementById('chart-container');
```

```
let voiceUsed = false;
```

```
const emojis = {
  toxic: "🤖", severe_toxic: "💣", obscene: "🚫",
  threat: "⚠️", insult: "😏", identity_hate: "🚫"
};
```

```
let barChart, pieChart;
```

```
let speakText = "";
```

```
function showChart() {
  const history = JSON.parse(localStorage.getItem('toxic_history') || '[]');
  if (!history.length) return alert("No data to chart.");
  const latest = history[history.length - 1].prediction;
  const labels = Object.keys(latest).map(k => k.replace('_', ' '));
  const values = Object.values(latest);

  if (barChart) barChart.destroy();
  if (pieChart) pieChart.destroy();
```

```
const barCtx = document.getElementById('barChart').getContext('2d');

barChart = new Chart(barCtx, {
  type: 'bar',
  data: {
    labels,
    datasets: [{
      label: 'Toxicity Scores',
      data: values,
      backgroundColor: values.map(v => v >= 0.5 ? 'red' : 'green')
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    scales: {
      y: { beginAtZero: true, max: 1 }
    },
    plugins: {
      legend: { display: false }
    }
  }
});
```

```
const pieCtx = document.getElementById('pieChart').getContext('2d');

pieChart = new Chart(pieCtx, {
  type: 'pie',
  data: {
    labels,
```

```
datasets: [{  
    data: values,  
    backgroundColor: ['red', 'orange', 'yellow', 'green', 'blue', 'purple']  
}]  
,  
options: {  
    responsive: true,  
    maintainAspectRatio: false,  
    plugins: {  
        legend: { position: 'bottom' }  
    }  
}  
});  
}
```

For run you code type: `python app.py`

Output

Not secure https://127.0.0.1:5000

Search, Star, Share, Verify that it's you, All Bookmarks

Gmail, Login - CodeTantra, KIET-MSERP, KIET-LMS, Pearson MePro, Documentation - M..., KIET, IMNEO KIET, kiet.cybervidya.net/...

Toxic Comment Classifier

Speak your Comment

"You are an idiot and so ugly!"

Check Check Another Comment

Result

identity hate: 13%	13%
insult: 71%	71%
obscene: 52%	52%
severe toxic: 5%	5%
threat: 5%	5%
toxic: 94%	94%

☒ This comment is toxic. Please be cautious!

Developed by Arun Kumar & Ayushi Saran Singh
for any query Contact on : thakurankumar200411@gmail.com
© Copyright :Toxic Comment Classifier-2025

